

I want to create a minecraft world where the surface is 30 blocks below build limit, with natural spawning of trees and structures, and most importantly, caves. I want stone and deep slate to be unbreakable like bedrock.



You can do this in Minecraft, but it's going to take a combination of:

1. **Custom world generation**
2. **A datapack or mod to make stone/deepslate unbreakable**
3. Optional tweaks for progression and ores

The good news is: your idea is absolutely possible.

What you want, technically

You want a world where:

- The **surface** is near the top of the world (about 30 blocks below build limit)
- Normal terrain still exists:
 - trees
 - villages
 - strongholds
 - caves
 - ores
- But:
 - **stone**
 - **deepslate**

behave like **bedrock** (cannot be mined)

That creates a “surface-only survival” style world where caves exist visually but can't be tunneled through.

Best way to do it

Option 1 — Easiest + Most Stable

Use a custom dimension datapack

Minecraft's worldgen system supports custom terrain height and cave generation through datapacks.  [Minecraft Wiki](#)

You would:

- Raise terrain generation upward
- Keep vanilla cave generation enabled
- Preserve vanilla structures

Recommended terrain settings

Vanilla 1.20+ world:

- Build limit: 320
- Sea level: 63

You want surface around:

- $Y \approx 290$

So your terrain should generate roughly:

- 250–295

with caves underneath.

The important part:

Keeping caves and structures

You **must NOT** use a void or flat generator.

Instead use:

- vanilla noise generation
- modified `noise_settings`

That preserves:

- caves
- aquifers

- mineshafts
- strongholds
- villages
- trees
- biome decoration

How to make stone/deepslate unbreakable

This is the hard part.

Minecraft does NOT let you directly assign “bedrock hardness” to vanilla blocks with datapacks alone.

You need either:

A mod (best option)

Best approach:

A small Fabric mod using block tags/events.

You can intercept block breaking and cancel it for:

- `minecraft:stone`
- `minecraft:deepslate`
- stone variants if desired

This gives true “unbreakable” behavior.

Simpler alternative:

Adventure-mode style datapack

A datapack can constantly prevent breaking certain blocks using tags/functions.

Examples exist for “unbreakable blocks” systems.  [CurseForge](#) +1

But this has downsides:

- less reliable
- can feel janky

- may conflict with mods
- creative-mode bypasses it

For your idea, a lightweight mod is much cleaner.

Recommended stack

If you want the cleanest experience

Use:

- Fabric loader
- Custom worldgen datapack
- Tiny custom mod

How I would structure the world

Terrain

Generate terrain between:

Layer	Y Level
Build limit	320
Surface	280–290
Major caves	220–280
Deep caves	100–220
Void floor	below

This creates gigantic underground void spaces beneath civilization.

Important design issue:

Ore accessibility

If stone is unbreakable:

- all ores trapped in stone become inaccessible

So you'll probably want one of these:

Option A

Expose ores in cave walls only

Option B

Generate ore blobs made of:

- gravel
- tuff
- calcite

instead of stone.

Option C

Use custom structures/mines for resources.

Easiest implementation path

Use an existing cave-world datapack as a base

Something like:

- Better Cave Worlds

already preserves:

- caves
- biomes
- structures

and can be modified upward.  Modrinth

The exact system I'd recommend

1. Create custom noise settings

Change:

- min_y
- height
- terrain density

to shift terrain near build height.

2. Keep vanilla biome source

This preserves:

- forests
- trees
- villages
- structures

3. Add Fabric mod

Cancel breaking for:

- stone
- deepslate

Pseudo-code:

Java

```
if(block == Blocks.STONE || block == Blocks.DEEPSLATE) {  
    cancelBreak();  
}
```

Cool side effects of your concept

Your world becomes:

- extremely vertical
- traversal-focused
- cave exploration without mining
- structure-loot dependent

- almost like a natural Skyblock

It's actually a very strong survival concept.

One warning

Strongholds and ancient cities may partially generate inside unbreakable terrain.

You may need:

- larger caves
- increased cave frequency
- exposed structure generation

to keep them accessible.

If you want, I can also help you with:

- a full datapack folder structure
- exact JSON worldgen files
- Fabric mod code
- terrain settings
- cave density tuning
- ore redistribution
- structure accessibility fixes
- making ONLY stone unbreakable while ores remain mineable
- making caves larger/more cinematic

That part gets pretty technical, but it's very doable.



Sources

Can I do all of this with the fabric mod, so that I can just have one thing to make rather than multiple different things?



Yes. A single Fabric mod is the cleanest way to do this.

You can handle **all of it** inside one mod:

- custom world generation
- high-altitude surface
- vanilla structures
- trees
- caves
- unbreakable stone/deepslate
- ore tweaks
- cave size changes
- progression systems

You do *not* need a separate datapack unless you want easier tweaking.

Recommended approach

Use Fabric + custom chunk generator settings

Modern Minecraft worldgen is data-driven internally, but Fabric mods can inject or register all of those settings directly.

Your mod can:

- register a custom dimension
- override noise settings
- create a custom world preset
- modify terrain density
- keep vanilla cave carvers
- keep vanilla structures

So from the player perspective:

Install mod → create world → select preset → done.

What your mod would actually contain

1. Custom world preset

Adds something like:

- "Sky Depths"
- "Ceilingworld"
- "Surfacebound"

to the world creation screen.

This preset uses custom noise settings.

2. Raised terrain generation

You'd shift terrain upward so the playable surface is near Y=290.

Conceptually:

```
Old terrain:  
64 = surface
```

```
Your terrain:  
290 = surface
```

while preserving normal cave carving below.

3. Vanilla cave generation

You KEEP:

- noise caves
- spaghetti caves
- aquifers
- ravines
- lush caves
- dripstone caves

So the underground still feels vanilla.

4. Structures and trees

If you use vanilla biome generation:

you automatically keep:

- villages
- woodland mansions
- strongholds
- ruined portals
- trees
- vegetation
- ores

This is the easy part.

5. Make stone/deepslate unbreakable

This is much easier in a Fabric mod than datapacks.

You can intercept block breaking events.

For example:

Java

```
PlayerBlockBreakEvents.BEFORE.register((world, player, pos, state, blockEntity) -> {
    Block block = state.getBlock();

    if(block == Blocks.STONE || block == Blocks.DEEPSLATE) {
        return false;
    }

    return true;
});
```

That alone blocks mining.

Better version

You probably want:

Allow ores to break

Otherwise resources become impossible.

So instead:

Unbreakable:

- stone
- granite
- diorite
- andesite
- deepslate
- tuff

Breakable:

- coal ore
- iron ore
- diamonds
- etc.

Even cooler option

Instead of making stone literally unbreakable:

Make it instantly regenerate

That preserves:

- explosions
- cave-ins
- mod compatibility

while preventing tunneling.

Very fun for gameplay.

Cave generation tuning

Since caves are now the *main* explorable space, you probably want:

- larger caves
- more cave frequency
- giant caverns
- more exposed surfaces

This can all be adjusted in the same mod.

One important technical note

World height limits

Minecraft hardcaps at:

- -64 bottom
- 320 top

So if your surface is at 290:

you only get ~30 blocks of “sky.”

That’s fine aesthetically, but:

- mountains may clip
- trees may hit build limit

So you’ll probably want:

- flatter terrain
- reduced mountain generation

Suggested world shape

A really good configuration:

Layer	Height
Bedrock ceiling feel	320
Surface	270–285

Layer	Height
Main caves	120–260
Deep caverns	-20–120
Bedrock floor	-64

This gives huge underground space.

Fabric APIs you'll likely use

Core

- [Fabric API](#)

Worldgen

- biome modification API
- chunk generator hooks
- noise settings registration

Event handling

- block break events

The easiest development route

Start from:

Fabric Example Mod

[Fabric Example Mod Github](#)

Then add:

1. custom world preset
2. block break prevention
3. terrain config

Difficulty estimate

Moderate, not extreme

Easy:

- prevent mining
- add preset
- preserve structures

Medium:

- terrain shifting
- cave tuning

Hard:

- polished balancing
- ore accessibility
- making structures always reachable

My recommendation

For your specific idea:

Make only “host rock” unbreakable

Meaning:

- stone variants unbreakable
- ores breakable
- dirt breakable
- gravel breakable

That preserves survival gameplay while still preventing strip mining.

It creates a genuinely unique exploration-focused Minecraft mode.



Since we are doing a mod, there's two more ideas I wanted

to explore. Firstly passive mobs spawn in flocks of 20-30, and always flee from players, and they try to stay grouped together, so more herd like behavior.



That's very achievable in a Fabric mod, and honestly fits your world concept extremely well. You're basically turning Minecraft into a hostile ecology simulator instead of a mining game.

What you want breaks down into three systems:

1. **Large herd spawning**
2. **Group cohesion behavior**
3. **Fear/avoidance AI**

All can be done in Fabric without replacing vanilla mobs entirely.

1. Large herd spawning

Vanilla passive mobs usually spawn in tiny packs:

- cows: 2-4
- sheep: 2-4
- pigs: 1-3

You can override biome spawn settings and increase pack size.

Example concept:

Java

```
new SpawnSettings.SpawnEntry(  
    EntityType.SHEEP,  
    weight,  
    20, // min  
    30  // max  
);
```

So instead of:

- scattered animals

you get:

- massive roaming herds.

Important balancing note

20–30 entities is a LOT for vanilla AI.

So you probably want:

Mob	Suggested Herd Size
Sheep	15–25
Cows	10–20
Pigs	8–16
Chickens	20–40
Horses	6–12

Otherwise TPS can suffer.

2. Herd behavior

Vanilla mobs technically “group” a little, but not intelligently.

You want:

- flocking
- regrouping
- movement alignment
- separation
- following leaders

This is basically a simplified **boids** system.

Simplest implementation

Every few ticks:

- find nearby same-type mobs
- move toward local center
- avoid getting too close
- follow herd direction

Pseudo-logic:

Java

```
Vec3 center = averagePositionOfNearbyHerd();  
moveToward(center);
```

Better implementation:

Leader-based herds

Choose one mob as:

- herd leader

Others:

- stay near leader
- mirror movement
- flee together

This is MUCH cheaper computationally.

And honestly looks more natural in Minecraft.

3. Fleeing from players

This is the easiest part.

Vanilla already has:

- `FleeEntityGoal`

You can massively increase:

- detection range

- panic duration

Example:

Java

```
new FleeEntityGoal<>(  
    this,  
    PlayerEntity.class,  
    32.0F,  
    1.4D,  
    1.8D  
);
```

Meaning:

- detect player from 32 blocks
- sprint away immediately

What makes this REALLY interesting

Because mining is impossible:

Animals become critical resources.

So:

- hunting becomes harder
- stealth matters
- terrain navigation matters

It naturally creates:

- tracking gameplay
- migration gameplay
- food scarcity

Very survival-heavy.

Emergent gameplay you'll get

Huge stampedes

If one mob panics:

- nearby herd panics
- whole group runs

This looks AWESOME in large cave systems.

Predatory feeling

Players stop feeling like:

- owner of the world

and start feeling like:

- invasive predator

Very cool atmosphere.

Performance considerations

This matters a LOT.

Your world already has:

- huge cave generation
- elevated terrain

Adding:

- 30-mob AI groups

can become expensive.

Best optimization approach

Use “virtual herds”

Instead of:

- every animal thinking independently

Use:

- one leader brain
- followers mirror behavior

That reduces AI cost enormously.

Recommended behavior stack

Passive mobs should:

Idle

- wander slowly
- stay grouped

Alert

- look toward disturbance
- tighten formation

Panic

- flee together
- avoid cave edges
- avoid players aggressively

Recover

- regroup after fleeing

You can go even further

This system supports:

Migration

Herds slowly travel biome-to-biome.

Sleeping behavior

Animals gather in safer open areas at night.

Predator/prey systems

Wolves scare herds from very far away.

Sound propagation

One frightened animal alerts others.

Technical recommendation

Don't rewrite vanilla entities.

Instead:

- inject goals into existing mobs

Using mixins/events.

That keeps:

- compatibility
- breeding
- drops
- pathfinding
- animations

Best Fabric tools for this

Core

- [Fabric API](#)

AI modification

Mixin system:

- [SpongePowered Mixin Wiki](#)

The biggest challenge

Not spawning.

Not fleeing.

Pathfinding in giant caves.

Vanilla pathfinding struggles with:

- cliffs
- verticality
- ravines

So herds may:

- split apart
- get stuck
- fall

You may eventually want:

- custom navigation
- terrain safety checks

But version 1 can absolutely work with vanilla navigation.

This overall design is becoming a genuinely distinctive survival overhaul:

- no mining
- cave traversal
- ecological hunting
- herd behavior
- resource scarcity
- vertical exploration

It's a strong gameplay identity already.



Polsia

Sponsored



Your AI cofounder for when you have no team

Code, outreach, content, ops — a team of AI agents on your business, and you can watch live

okay, I haven't made mods before so I want to keep things simple. honestly, I don't want to spend more than 20h making this mod. I'll be using AI to make it for me.



That changes the strategy completely — and in a good way.

With a 20-hour budget and AI assistance, you should avoid:

- custom pathfinding
- custom chunk generators from scratch
- rewriting mob AI systems
- advanced worldgen math

You want:

- minimal code
- maximum gameplay impact
- mostly vanilla systems with tweaks

That is VERY doable.

Revised scope recommendation

Keep these features

1. High-altitude world

YES

2. Unbreakable stone/deepslate

YES

3. Large fleeing herds

YES

These are achievable in a first mod.

Cut or simplify these

DON'T:

- implement true flocking AI
- make custom navigation
- create migration systems
- rewrite cave generation

Those become rabbit holes fast.

Instead:

Fake the herd behavior

This is the key simplification.

You can get 80% of the feeling using:

- large spawn groups
- stronger flee radius
- slightly reduced wandering

That alone creates:

- clustered animals
- stampedes
- skittish wildlife

without writing complicated AI.

Your realistic architecture

ONE Fabric mod

Containing:

Feature	Difficulty
Custom world preset	Medium

Feature	Difficulty
Raise terrain upward	Medium
Prevent stone mining	Easy
Increase herd size	Easy
Increase flee radius	Easy

That is genuinely achievable.

Simplest possible implementation

Phase 1 (2–4 hours)

Set up Fabric mod.

Use:

- Java
- IntelliJ
- Fabric example mod

Start here:

[Fabric Example Mod Github](#)

Phase 2 (1–2 hours)

Make stone unbreakable

This is probably your easiest win.

AI can generate this almost perfectly.

You mainly need:

- block break event
- block whitelist/blacklist

Phase 3 (2–3 hours)

Increase herd sizes

Use biome spawn modification.

Again:

- straightforward
- AI-friendly
- lots of examples online

Phase 4 (2–3 hours)

Add flee behavior

Inject:

- `FleeEntityGoal`

into passive mobs.

Very manageable.

Phase 5 (hardest part: 6–10 hours)

World generation

This is the only dangerous feature scope-wise.

IMPORTANT:

Do NOT write a chunk generator from scratch.

Instead:

Modify vanilla noise settings.

That preserves:

- caves
- structures
- biomes
- trees

without huge complexity.

Even smarter approach

Honestly?

For a 20-hour project:

Use a datapack **INSIDE** the mod.

You can bundle datapack-style worldgen JSON inside the mod jar.

This is VERY common in modern Fabric mods.

Advantages:

- almost no Java code
- easier for AI to generate
- easier to tweak terrain
- less likely to crash

You still ship:

- ONE MOD FILE

So the user experience remains:

install mod and play

This is probably your ideal setup

Java handles:

- mining prevention
- mob behavior

Built-in datapack handles:

- terrain height
- caves
- structures

That is dramatically easier.

Extremely important advice

Don't start with the worldgen.

Start with:

1. unbreakable stone
2. animal spawning
3. flee AI

Get a working mod FIRST.

Then do terrain.

Because worldgen debugging can consume your entire 20 hours.

The easiest possible worldgen trick

Instead of fully custom terrain:

Raise sea level and offset terrain upward.

You can get surprisingly close to your vision with relatively small noise tweaks.

You do NOT need a revolutionary generator.

Your MVP (minimum viable product)

If I were timeboxing this:

Required:

- elevated terrain
- caves preserved
- stone unbreakable
- larger passive mob groups
- flee AI

Optional:

- deeper cave tuning
- custom ores
- giant caverns
- advanced herd logic

AI-assisted development workflow

This matters a lot.

Good AI prompts:

Instead of:

“make my whole mod”

Do:

“In Fabric 1.21, how do I cancel block breaking for stone and deepslate using PlayerBlockBreakEvents?”

Small prompts work MUCH better.

Best workflow

Build feature-by-feature

1. Verify compile
2. Launch game
3. Test
4. Commit
5. Next feature

Do NOT let AI generate:

- entire architecture
- huge multi-file systems

That's where hallucinations explode.

Your biggest actual risk

Not coding.

Minecraft version mismatch.

Fabric modding changes constantly.

So:

- choose ONE Minecraft version
- use matching Fabric API/docs/examples

I strongly recommend:

- latest stable Fabric version
- NOT snapshots

Overall assessment

Your project is realistically achievable in 20 hours IF you:

- keep AI prompts narrow
- avoid custom pathfinding
- avoid custom chunk generators
- fake complex behaviors with vanilla systems
- use datapack-style worldgen bundled into the mod

That's a very reasonable first mod.



Okay one last thing I want to add. I want to add a custom

unbreakable ore that generates a small random amount of RF.



That's actually a *great* final feature for this mod, because it fits your gameplay loop and is much simpler than it sounds.

And importantly:

- it's self-contained
- low code complexity
- high gameplay value

This is exactly the kind of "big-feeling" feature you want in a 20-hour project.

Your design now

You've basically created:

"Surface Survival"

Where:

- mining is impossible
- caves are traversal spaces
- resources are exposed naturally
- animals are difficult to hunt
- progression depends on exploration

So adding:

naturally occurring RF-generating ore

gives players:

- energy infrastructure
- exploration rewards
- cave objectives

without traditional mining.

Very cohesive design.

Keep this SIMPLE

DO NOT:

- make a whole tech mod
- implement machines
- add power networks

Just:

- create a block
- make it emit RF passively

That's enough.

The easiest implementation

Add a block entity

Your ore block:

- cannot be mined
- periodically generates RF
- exposes Forge Energy compatibility (or TeamReborn Energy if Fabric-native)

IMPORTANT:

RF itself is Forge terminology

On Fabric, the modern equivalent is usually:

Energy API

from:

- Team Reborn Energy
- or
- TechReborn ecosystem

Most Fabric tech mods support it.

Simplest practical solution

Use:

TeamReborn Energy API

[TeamReborn Energy Github](#)

This gives:

- Fabric-compatible power system
- simple interfaces
- broad mod compatibility

Your ore concept

Example behavior

“Resonant Ore”

- unbreakable
- naturally generates in caves
- slowly emits energy
- requires infrastructure nearby

Keep generation EXPOSED

Since players can't mine:

- the ore **MUST** generate visibly.

Meaning:

- cave walls
- cave ceilings
- exposed cliffs

NOT buried in stone.

VERY important simplification

Don't make it "random ticking."

Instead:

- generate fixed RF/t

Example:

- 4 RF/t
- 8 RF/t
- random per block at placement time

Much easier.

Simple implementation idea

When ore generates:

- assign:
 - 2–16 RF/t randomly

Store value in block entity NBT.

So each node feels unique.

Why this is good

Players naturally:

- search caves
- map ore locations
- build infrastructure around nodes

instead of strip mining.

It reinforces your entire game design.

Difficulty estimate

Feature

Difficulty

Feature	Difficulty
Custom block	Easy
Ore generation	Medium
Unbreakable behavior	Already solved
RF generation	Medium
Compatibility	Medium

Totally manageable.

EASIEST possible implementation

Honestly?

Skip true RF compatibility initially.

Version 1:

- block entity stores “energy”
- comparator output reflects strength
- particles/sounds

Just fake it.

Then later:

- add proper energy API support.

This saves HOURS.

If you DO want real energy compatibility

The simplest architecture:

Ore block entity:

Implements:

Java

EnergyStorage

Then:

- every tick:
 - increase stored power
- adjacent blocks can extract.

Worldgen recommendation

Since this is your key resource:

Make it uncommon but visible.

Example:

- 1–3 veins per chunk
- 1 block veins
- always exposed to air

This creates:

- exploration incentives
- territorial gameplay

VERY important:

Don't generate too many.

Because players can't remove them.

Too many permanent ticking block entities can hurt performance.

Recommended limits

Good target:

- 1 node every 2–4 chunks

NOT:

- giant ore veins

Your total mod scope now

Still reasonable:

System	Complexity
Unbreakable stone	Easy
Elevated worldgen	Medium
Herd spawning	Easy
Flee AI	Easy
RF ore	Medium

Still viable in ~20h with AI assistance if you:

- keep implementation simple
- avoid overengineering
- use existing APIs
- don't chase polish endlessly.

This is now a legitimately compelling survival overhaul concept, not just a gimmick mod.



I don't want it to store any energy, just a small amount of RF/tick, and the player has to capture that somehow to actually make use of it, and that stuff I'll borrow from other mods.



That's much better design-wise *and* much easier technically.

You basically want:

- an infinite passive energy source
- no internal buffer
- no machines
- no storage logic
- just:
 “this block outputs X RF/t forever”

That is a very clean mechanic.

This dramatically simplifies implementation

You do NOT need:

- ticking block entities
- internal energy storage
- save data
- capability syncing
- machine logic

Instead:

- the block itself simply exposes energy extraction.

Conceptually

Your ore behaves like:

“a naturally occurring geothermal vent”

Players:

- find exposed nodes
- build collectors nearby
- route power elsewhere

Very elegant.

Even better:

Randomize output ON GENERATION

Example:

- 1–8 RF/t
- 4–16 RF/t

This makes:

- some nodes valuable
- exploration meaningful

without complexity.

Simplest architecture

Custom block entity:

Contains ONLY:

Java

```
int generationRate;
```

No storage.

No ticking.

No accumulating energy.

Then implement:

Java

```
extractEnergy(...)
```

When another mod tries to pull power:

- provide up to `generationRate`

That's it.

Important consequence

This means:

- power cannot be buffered inside ore
- unused energy is wasted

Which is actually REALLY good gameplay.

Players need:

- infrastructure
- continuous consumption
- power routing

Why this is ideal for your mod

Because your world is exploration-focused.

So progression becomes:

1. Find cave node
2. Secure area
3. Build infrastructure
4. Route power upward

instead of:

- strip mine
- quarry
- automate everything instantly

Even better:

Make the ore indestructible too

This creates:

- permanent world resources
- territorial gameplay
- infrastructure hubs

Very cool emergent behavior.

Performance benefits

Huge improvement.

Because:

- no ticking
- no storage updates
- no synchronization spam

You can have many nodes safely.

The easiest implementation path

Use a simple energy API interface

For Fabric:

- TeamReborn Energy API

Your block entity simply:

- reports available extraction
- refuses insertion

Done.

You probably DON'T even need ticking code

That's the beauty here.

Energy availability is:

- effectively virtual
- computed on demand

Very lightweight.

Recommended numbers

Keep values LOW.

Because infinite passive power scales hard.

Good:

- 2 RF/t
- 4 RF/t
- rare 8 RF/t

Dangerous:

- 50+ RF/t

Low generation encourages:

- many exploration sites
- distributed infrastructure

which matches your mod perfectly.

Strong recommendation:

Keep ore exposed to air only

Otherwise players can't access it.

Generate:

- cave walls
- cave ceilings
- cliff faces

Never embedded fully in stone.

One VERY important compatibility note

Since you said:

"I'll borrow the infrastructure from other mods"

you should choose ONE ecosystem.

For Fabric:

Best choice is probably:

- TechReborn energy ecosystem

because it's mature and simple.

Your final mod scope now feels VERY realistic

Honestly, this is now a pretty achievable first mod because every system is conceptually simple:

Feature	Complexity
Elevated world	Medium
Preserve caves/structures	Medium
Unbreakable stone	Easy
Large passive herds	Easy
Flee AI	Easy
Infinite RF ore	Medium

None of these individually are monstrous anymore.

The biggest time sink will still be:

world generation tuning.

Everything else is pretty approachable with AI assistance.



